EPOKA UNIVERSITY
CEN 350 — Theory of Computation

Nested Structure Parser with Pushdown Automata

Mathematical Expressions — Mixed Delimiters with Escape Support

Project Code:	**93**
Domain (93 mod 10):	**3 — Mathematical Expressions**
Constraint ((93÷10) mod 5):	**4 — Mixed terminal symbols**
Feature (93 mod 7):	**2 — Escape support**
Implementation:	**Option B — Python PDA Simulator**
Final Submission:	**May 25, 2026**

AUTHORS

Xhule Metaj
xmetaj23@epoka.edu.al

Elvis Sula
esula@epoka.edu.al

1. Project Overview

This project models a restricted subset of mathematical expression syntax using a Pushdown Automaton (PDA). The target language consists of strings whose mixed delimiters — parentheses `( )`, square brackets `[]`, and curly braces `{}` — are properly nested. A backslash escape sequence allows delimiter characters to appear as literals.

★ Goal: Demonstrate understanding of Pushdown Automata and context-free languages, NOT to build a full production parser. Scope is intentionally restricted.

Parameter	Formula	Value
Domain	`93 mod 10 = 3`	Mathematical Expressions
Constraint	`(93 ÷ 10) mod 5 = 4`	Mixed terminal symbols with nested structures
Feature	`93 mod 7 = 2`	Support escaped symbols (\x)

2. Formal Language Specification

2.1 Informal Semantics

Delimiter matching: Each closing delimiter must match the most recent unmatched opening delimiter of the same type. Mixed nesting such as ([{}]) is allowed when properly matched.

Math content: Characters that are not delimiters (and not introduced by escape) may appear freely between delimiters: digits, letters, operators (+–*/. ^), and whitespace.

Escape sequences: If \ is read, the next symbol is consumed as a literal and does not affect the stack. For example \(is just a literal (, not an opener.

Empty string: The empty string is accepted (no unmatched delimiters exist).

2.2 Input Alphabet Σ

Category	Symbols
Open delimiters	`( [ {`
Close delimiters	`) ] }`
Escape	`\`
Digits	`0 – 9`
Letters	`a–z, A–Z, _`
Operators	`+ - * / . ^`
Whitespace	`space, tab`

Any character not in Σ causes immediate rejection.

2.3 Sample Valid and Invalid Strings

String	Expected	Notes

`(1+2)`	ACCEPT	Simple matched parentheses
`([{3.14}])`	ACCEPT	Deep mixed nesting (3 levels)
`a*(b+[c])`	ACCEPT	Mixed delimiters with math operators
`x+\(y\)`	ACCEPT	Escaped parentheses treated as literals
`(empty)`	ACCEPT	Boundary: no delimiters
`(1+2`	REJECT	Unclosed (— unmatched opener
`([)]`	REJECT	Mismatched:) found when [expected
`(1+2))`	REJECT	Extra) with empty delimiter stack
`(1@2)`	REJECT	Illegal symbol @ not in Σ

3. Context-Free Grammar (CFG)

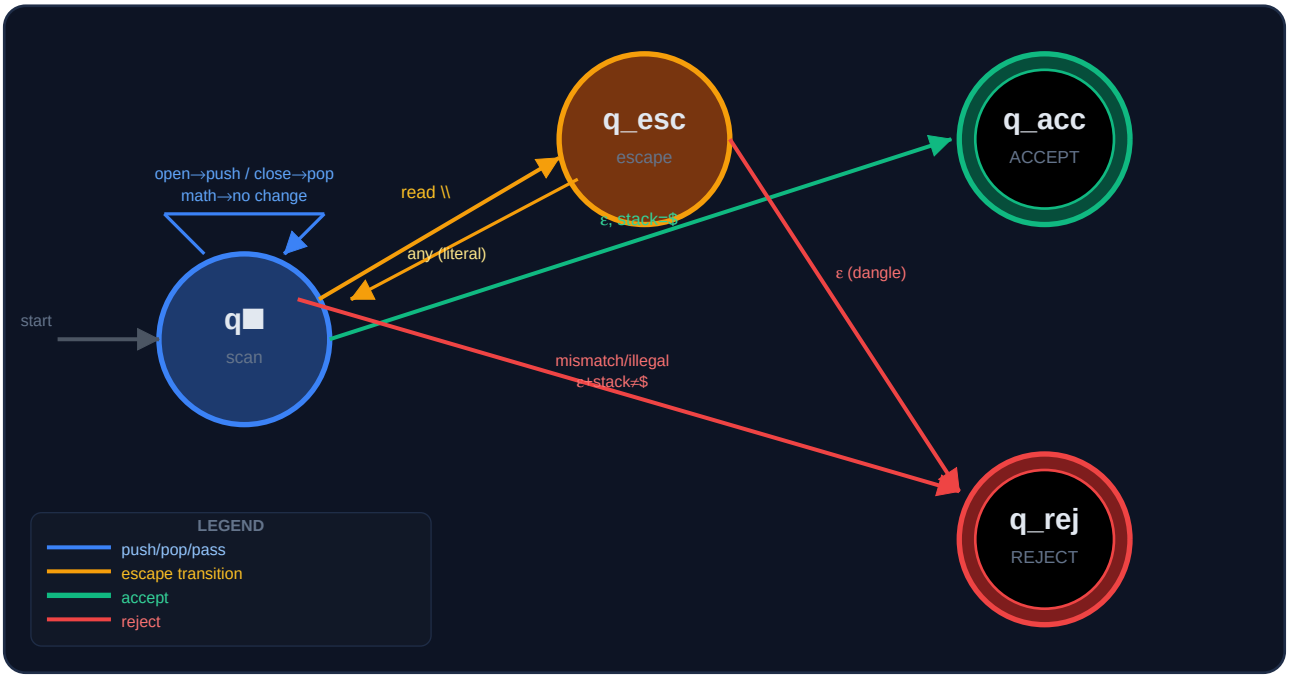
The CFG below generates exactly the language L. Non-terminals: S (start / expression sequence), T (single term), L (literal sequence), M (one math character), ESC (escape pair).

```
S → ε | T S
T → ( S ) | [ S ] | { S } | L
L → ε | M L
M → digit | letter | op | ws | ESC
ESC → \( | \) | \[ | \] | \{ | \} | \\
```

Notes: T S allows concatenation of delimiter groups and literal segments. ESC productions implement Feature 2 (escaped symbols). The three bracket types model Constraint 4 (mixed nested terminals). This CFG is context-free; a single-stack PDA suffices for delimiter nesting.

4. PDA Design

4.1 State Diagram



4.2 PDA Components

Component	Definition
States Q	`{ q0, q_esc, q_accept, q_reject }`
Input alphabet Σ	As defined in Section 2.2
Stack alphabet Γ	`{ $, (, [, { }`
Start state	`q0`
Start stack	`$ (bottom-of-stack marker)`
Accept state	`q_accept (reached when input empty and stack = {$})`
Reject state	`q_reject (reached on any invalid transition)`

4.3 Transition Function δ — From state q■

Input Symbol	Stack Top	Next State	Stack Action	Notes
`( or [ or {`	`any`	`q0`	`push d`	Open delimiter
`)`	`(`	`q0`	`pop (`	Matching close
`]`	`[`	`q0`	`pop [`	Matching close
`}`	`{`	`q0`	`pop {`	Matching close
`) or ] or }`	`wrong / $`	`q_reject`	—	Mismatch or empty stack
`\`	`any`	`q_esc`	`no change`	Enter escape mode
`digit,letter,op,ws`	`any`	`q0`	`no change`	Math terminal pass-through

ε (end)	$ only	q_accept	—	All delimiters matched
ε (end)	else	q_reject	—	Unmatched openers remain
illegal char	any	q_reject	—	Symbol not in Σ

4.4 Transition Function δ — From state q_esc

Input Symbol	Stack Top	Next State	Stack Action	Notes
any symbol	any	q0	no change	Consumed as literal; not treated as delimiter
ε (end)	any	q_reject	—	Dangling backslash — escape with no following symbol

5. Correctness Proof

Claim: PDA M accepts exactly the language L of strings whose mixed delimiters (), [], {} are properly nested, whose non-delimiter symbols belong to Σ, and whose escape sequences \x treat x as a literal.

5.1 Soundness — every accepted string ∈ L

> I Invariant I: At any step in q0, the stack (above $) is a sequence of opening delimiters appearing in the input prefix in order, each corresponding to an unmatched opener.

Base: Initially stack = {$}; no openers — invariant holds trivially.

Push: Reading ([{ appends the matching opener; invariant maintained.

Pop: Reading a closer removes the top only if it is the matching opener; otherwise → q_reject. We never pop a non-matching symbol; invariant preserved.

Escape: In q_esc, one character consumed with no stack change. Delimiters in escaped form cannot affect I.

Literals: Math characters do not alter the stack; invariant trivially maintained.

Conclusion: On acceptance, input is empty and stack = {$}, so every opener was matched. Illegal symbols cause rejection. Therefore every accepted string satisfies L.

5.2 Completeness — every w ∈ L is accepted

Delimiters: Each opener in w triggers a push. Since w is well-nested, when a closer appears the stack top is exactly its partner, so the PDA pops instead of rejecting.

Escape: Each pair \x moves to q_esc and consumes x without stack change — legal in L.

Math terminals: Characters only advance the read head, leaving the stack intact.

Termination: At end of input, no unmatched openers remain (w ∈ L), so stack = {$} and the PDA enters q_accept. Therefore w is accepted. ■

5.3 Rejection Cases

Condition	Cause
Closer ≠ stack top	Mismatched delimiter nesting
Closer, stack = {$}	Extra closing delimiter
Input ends, stack ≠ {$}	Unclosed opening delimiter(s)
`\ at end of input`	Dangling escape sequence
Symbol ∉ Σ	Illegal character (e.g. @)

5.4 Non-Context-Free Limitation

Our PDA handles Dyck-style mixed bracket matching — this is exactly context-free. However, requiring the total count of (to equal the total count of [globally (analogous to a^n b^n c^n) is beyond CFL and would require a Turing machine. Our design intentionally stays within the well-defined CFL boundary.

6. Test Cases with Execution Traces

Test Case 1 — ACCEPT: `(1+2)` — Simple valid

A simple expression with one matched pair of parentheses. Demonstrates the basic push/pop cycle.

Step	State	Remaining Input	Stack (bottom→top)	Action
0	q0	`(1+2)`	$	Start
1	q0	`1+2)`	$(	Read '(' → push '('
2	q0	`+2)`	$(	Read '1' → no stack change
3	q0	`2)`	$(	Read '+' → no stack change
4	q0	`)`	$(	Read '2' → no stack change
5	q0	ε	$	Read ')' → pop '('
6	q_accept	ε	$	Stack = $ → ACCEPT

Test Case 2 — ACCEPT: `([{3.14}])` — Deep mixed nesting

Three levels of nested mixed delimiters. Stack grows to depth 3 then collapses back to $.

Step	State	Remaining	Stack	Action
0	q0	`([{3.14}])`	$	Start
1	q0	`[{3.14}])`	$(	Read '(' → push '('
2	q0	`{3.14}])`	$([	Read '[' → push '['
3	q0	`3.14}])`	$([{	Read '{' → push '{'
4–7	q0	`}])`	$([{	Read '3','.','1','4' → no change
8	q0	`])`	$([	Read '}' → pop '{'
9	q0	`)`	$(	Read ']' → pop '['
10	q0	ε	$	Read ')' → pop '('
11	q_accept	ε	$	Stack = $ → ACCEPT

Test Case 4 — ACCEPT: `x+\(y\)` — Escaped delimiters

Demonstrates Feature 2: backslash causes q_esc transition; escaped (and) are consumed as literals without affecting the stack.

Step	State	Remaining	Stack	Action
0	q0	`x+\(y\)`	$	Start
1	q0	`+\(y\)`	$	Read 'x' → no change
2	q0	`\(y\)`	$	Read '+' → no change
3	q_esc	`(y\)`	$	Read '\' → enter escape state
4	q0	`y\)`	$	Escape: '(' consumed as literal (no push!)
5	q0	`\)`	$	Read 'y' → no change
6	q_esc	`)`	$	Read '\' → enter escape state
7	q0	ε	$	Escape: ')' consumed as literal (no pop!)
8	q_accept	ε	$	Stack = $ → ACCEPT

Test Case 6 — REJECT: `(1+2` — Incomplete / unclosed

Input ends with '(' still on the stack. Failure point: Step 5 — input exhausted with unmatched opener.

Step	State	Remaining	Stack	Action
0	q0	`(1+2`	`$`	Start
1	q0	`1+2`	`$(`	Read '(' → push '('
2	q0	`+2`	`$(`	Read '1' → no change
3	q0	`2`	`$(`	Read '+' → no change
4	q0	ε	`$(`	Read '2' → no change
5	q_reject	ε	`$(`	Input exhausted; unmatched ['('] on stack → REJECT

Test Case 7 — REJECT: `([)]` — Mismatched order

Classic cross-nesting error. Stack top is '[' but ')' is read — expects ']'. Failure at Step 3.

Step	State	Remaining	Stack	Action
0	q0	`([)]`	`$`	Start
1	q0	`[)]`	`$(`	Read '(' → push '('
2	q0	`)]`	`$([`	Read '[' → push '['
3	q_reject	`)]`	`$([`	Read ')' — top='[', expected ']' → REJECT

Test Case 8 — REJECT: `(1+2))` — Extra closing delimiter

After the first ')' pops '(', the stack is back to {$}. The second ')' finds no opener. Failure at Step 6.

Step	State	Remaining	Stack	Action
0	q0	`(1+2))`	`$`	Start
1	q0	`1+2))`	`$(`	Read '(' → push '('
2	q0	`+2))`	`$(`	Read '1' → no change
3	q0	`2))`	`$(`	Read '+' → no change
4	q0	`))`	`$(`	Read '2' → no change
5	q0	`)`	`$`	Read ')' → pop '('
6	q_reject	`)`	`$`	Read ')' — stack empty (only $) → REJECT

7. Implementation — Option B (Python PDA Simulator)

The project implements Option B: a Python program that directly simulates the PDA transition function, maintaining a stack and recording a step-by-step execution trace.

7.1 Project Structure

```
Xhule_Elvis_Project/
■■■ src/
■ ■■■ pda.py # MathNestedPDA class — PDA simulator
■ ■■■ main.py # CLI — accepts input string or --examples flag
■■■ tests/
■ ■■■ test_pda.py # unittest suite covering all 8 test cases
■■■ docs/
■■■ formal_specification.md
■■■ pda_design.md
■■■ correctness_proof.md
■■■ sample_traces.txt
```

7.2 How to Run

```
# Run all built-in test cases
python3 src/main.py --examples

# Validate a single expression with full trace
python3 src/main.py "(1+2)*[a+b]"

# Test escape sequences
python3 src/main.py 'x+\(y\)'

# Run unit tests
python3 -m unittest discover -s tests -v
```

7.3 Example Program Output

```
Input: "([{3.14}])"

Step State Remaining Stack Action
----------------------------------------------------------------------
0 q0 '([{3.14}])' $ Start
1 q0 '[{3.14}])' $( Read '(' → push '('
2 q0 '{3.14}])' $([ Read '[' → push '['
3 q0 '3.14}])' $([{ Read '{' → push '{'
... q0 ... ... Read '3','.','1','4' → no change
8 q0 '])' ' $([ Read '}' → pop '{'
9 q0 ')' $( Read ']' → pop '['
10 q0 '' $ Read ')' → pop '('
11 q_accept '' $ Input exhausted → Accept

Final decision: ACCEPT
```

7.4 Test Results Summary

Input	Expected	Got	Status

`(1+2)`	ACCEPT	ACCEPT	PASS ✓
`([{3.14}])`	ACCEPT	ACCEPT	PASS ✓
`a*(b+[c])`	ACCEPT	ACCEPT	PASS ✓
`x+\(y\)`	ACCEPT	ACCEPT	PASS ✓
`(empty / ws)`	ACCEPT	ACCEPT	PASS ✓
`(1+2`	REJECT	REJECT	PASS ✓
`([)]`	REJECT	REJECT	PASS ✓
`(1+2))`	REJECT	REJECT	PASS ✓

All 8 test cases pass. The PDA correctly accepts exactly the strings in L and rejects all strings outside L as specified.